19. Profiling, Race Detector, Performance Mental Model

19. Profiling, Race Detector, Performance Mental Model

Phase: Phase 3 — Idioms and engineering judgment
Estimated time: 1–2 days
Slug: `19-profiling-performance`

Learning objectives

- Run the race detector and interpret failures
- Capture CPU and memory profiles with pprof
- Reason about allocations and escape analysis at a high level
- Optimize only with evidence

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Guessing at performance wastes time. pprof, the race detector, and a simple mental model of allocations keep optimizations honest.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- `go test -race` finds data races.
- pprof profiles CPU, heap, goroutines, blockers.
- Avoid premature optimization; measure first.
- Allocations and interface/boxed conversions show up in profiles.
- Benchmark before and after with `go test -bench`.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-profiling, && cd /tmp/gw-profiling,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func Sum(xs []int) int {
    s := 0
    for _, v := range xs {
        s += v
    }
    return s
}

func main() {
    xs := make([]int, 1000)
    for i := range xs {
        xs[i] = i
    }
    fmt.Println(Sum(xs))
}
```

Run it:

```
go run .
```

Expected output:

```
499500
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Write a BenchmarkSum and run -bench=.
2. Introduce a deliberate race and catch it with -race.
3. Read `go tool pprof` help once (even if you only skim).

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Race detector

```
go test -race ./...
```

Detects concurrent reads/writes without synchronization. **Use it in CI.**

pprof

```
go test -bench=. -cpuprofile=cpu.out
go tool pprof cpu.out
```

Or expose `/debug/pprof` on a debug port in services (carefully locked down).

Mental model

1. Correctness first
2. Measure
3. Fix algorithmic issues before micro-opts
4. Reduce allocations in hot loops (preallocate slices, avoid string concat in loops)

Escape analysis decides heap vs stack; pointers and interfaces often force heap allocation.

Failure modes to watch while coding

- Optimizing without benchmarks
- Disabling race detector because “it’s slow” in CI forever
- Caching everything without eviction strategy

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- cProfile ≈ pprof; Go’s is tightly integrated with the runtime.
- GIL hides some races; Go encourages finding them with -race.
- Microbenchmarks lie without careful b.N loops — same in both languages.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Fix algorithmic complexity before micro-caching. O(n^2) beats any pprof tweak.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Profiling, Race Detector, Performance Mental Model” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Optimizing without benchmarks
- Disabling race detector because “it’s slow” in CI forever
- Caching everything without eviction strategy

Exercises

Work in `exercises/19-profiling-performance/`. Read that folder’s `README.md` before you code.

What you will build

Implement the sum/perf stub correctly; tests may check results more than timing.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/19-profiling-performance/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does -race detect?
2. Name two profile types
3. Why measure before optimizing?

Further reading

- https://go.dev/blog/pprof
- https://go.dev/doc/articles/race_detector